DCC042 – Robótica Móvel

Roteiro de Laboratório 2 — TurtleBot3

Odometria e construção de um mapa métrico com LiDAR

Este roteiro está dividido em duas partes: (1) preparação, a ser feita ANTES da aula de laboratório, incluindo a implementação e o teste das funções que serão usadas em sala; e (2) a atividade prática, realizada com o robô físico.

> **Antes de vir para o laboratório**
>
> Complete integralmente a Parte 1 em seu próprio computador. As funções implementadas na preparação são a base de toda a atividade em sala. Quem chegar sem elas prontas e testadas dificilmente concluirá o laboratório no tempo de aula.

Parte 1 — Preparação (fazer antes da aula)

1.1 Visão geral

No Laboratório 1 você enviou comandos de velocidade e leu o LiDAR. Agora vamos usar esses dados para responder a duas perguntas: (a) onde o robô acha que está? e (b) como é o ambiente ao redor dele? Em sala você irá: calcular a pose do robô por conta própria a partir dos encoders das rodas, comparar o resultado com a odometria publicada pelo robô no tópico /odom, e usar a pose para transformar as leituras do LiDAR para o referencial global, construindo um mapa métrico simples formado por pontos (uma nuvem de pontos, e não uma grade de ocupação).

1.2 Requisitos

- Ambiente do Laboratório 1 funcionando (Python 3 e roslibpy).
- Bibliotecas adicionais: numpy e matplotlib.
- Material das aulas de Descrição Espacial, Transformações Homogêneas e Locomoção (modelos cinemáticos).

```
pip install numpy matplotlib
```

1.3 Revisão teórica

Odometria de um robô de acionamento diferencial. Sejam r o raio das rodas, L a distância entre elas, e Δϕ_E e Δϕ_D as variações angulares (em radianos) das rodas esquerda e direita entre duas leituras consecutivas dos encoders. Então:

Atenção: os valores de r e L NÃO serão fornecidos. Você mesmo os medirá no robô, em sala de aula (seção 2.2). Na preparação, use apenas os valores fictícios dos testes abaixo.

```
d_E = r * dphi_E          # deslocamento da roda esquerda
d_D = r * dphi_D          # deslocamento da roda direita
ds     = (d_D + d_E) / 2  # deslocamento do centro do robô
dtheta = (d_D - d_E) / L  # variação de orientação

x     = x + ds * cos(theta + dtheta/2)
y     = y + ds * sin(theta + dtheta/2)
theta = theta + dtheta
```

Transformação do LiDAR para o referencial global. Um feixe com distância r_i e ângulo α_i (no referencial do robô) corresponde ao ponto (r_i cos α_i, r_i sin α_i) no referencial do robô. Dada a pose (x, y, θ) do robô no mundo, esse ponto vai para o referencial global por uma rotação de θ seguida de uma translação de (x, y). Esta é exatamente a transformação homogênea 2D vista em aula.

1.4 Implementação e teste das funções (sem o robô)

Crie um arquivo chamado lab2_utils.py com as três funções abaixo e complete os trechos marcados com TODO. Em seguida, rode os testes: todos os asserts devem passar antes de você vir para o laboratório.

```
# lab2_utils.py
import math
import numpy as np

def normaliza_angulo(a):
    """Leva o ângulo para o intervalo (-pi, pi]."""
    return math.atan2(math.sin(a), math.cos(a))

def atualiza_odometria(x, y, theta, dphi_e, dphi_d, r, L):
    """Integra um passo de odometria. Retorna a nova pose (x, y, theta)."""
    # TODO
    raise NotImplementedError

def scan_para_mundo(ranges, angle_min, angle_increment, pose,
                    range_min=0.12, range_max=3.5, passo=1):
    """Converte um scan em pontos (N x 2) no referencial global.
    - descarte leituras inválidas (inf, nan, fora de [range_min, range_max])
    - use apenas 1 a cada 'passo' feixes
    - pose = (x, y, theta) do robô no mundo
    """
    # TODO
    raise NotImplementedError
```

Testes (cole no final do arquivo ou em uma célula separada):

```
from lab2_utils import *

# Valores FICTÍCIOS, usados só nos testes (os reais serão medidos em sala)
R, L = 0.05, 0.20

# Teste 1: ambas as rodas giram 1 rad -> robô anda em linha reta R*1
x, y, t = atualiza_odometria(0, 0, 0, 1.0, 1.0, R, L)
assert abs(x - R) < 1e-6 and abs(y) < 1e-6 and abs(t) < 1e-6

# Teste 2: rodas em sentidos opostos -> gira sem sair do lugar
x, y, t = atualiza_odometria(0, 0, 0, -1.0, 1.0, R, L)
assert abs(x) < 1e-6 and abs(y) < 1e-6
assert abs(t - 2 * R / L) < 1e-6

# Teste 3: robô em (1, 0) virado para +y; feixe frontal de 1 m
# deve atingir o ponto (1, 1) no mundo
pts = scan_para_mundo([1.0], 0.0, 0.01, (1.0, 0.0, math.pi/2))
assert pts.shape == (1, 2)
assert abs(pts[0, 0] - 1.0) < 1e-6 and abs(pts[0, 1] - 1.0) < 1e-6

# Teste 4: leituras inválidas devem ser descartadas
pts = scan_para_mundo([float('inf'), 0.0, 1.0], 0.0, 0.1, (0, 0, 0))
assert pts.shape == (1, 2)

print('Todos os testes passaram!')
```

Dica

No Teste 3, desenhe a situação no papel antes de rodar o código. Se o seu resultado for (1, -1) ou (-1, 1), há um erro de sinal ou de ordem na rotação.

1.5 Teste de conexão (sem o robô)

Reexecute o script de teste de conexão do Laboratório 1 e confirme que o roslibpy continua funcionando. Como antes, sem o robô na rede é esperado que a conexão falhe.

Checklist antes de sair de casa

[ ] numpy e matplotlib instalados [ ] lab2_utils.py com as três funções implementadas [ ] todos os testes passando [ ] notebook carregado e com bateria

Parte 2 — Atividade prática em laboratório

2.1 Objetivo

Você irá: (1) medir os parâmetros geométricos do robô e calcular a odometria a partir das posições angulares das rodas; (2) comparar visualmente a sua odometria com a posição publicada pelo robô em /odom; (3) construir um mapa métrico (nuvem de pontos) transformando as leituras do LiDAR para o referencial global; (4) analisar como os erros de odometria afetam o mapa.

2.2 Informações fornecidas em sala

- IP do robô do seu grupo e porta 9090.
- Se for o caso, o prefixo de namespace dos tópicos do seu robô.

Sugestão de tempo (100 min): medição dos parâmetros – 10 min; Etapa 1 – 10 min; Etapa 2 – 20 min; Etapa 3 – 15 min; Etapa 4 – 35 min; fechamento – 10 min.

2.2.1 Medição dos parâmetros geométricos do robô (obrigatória, antes de tudo)

Os parâmetros NÃO são fornecidos

O raio das rodas (r) e a distância entre as rodas (L) devem ser MEDIDOS por vocês, no robô do seu grupo, no início da aula. Não use valores de catálogo, da internet ou de outros grupos: robôs diferentes e desgaste podem gerar valores diferentes, e é justamente a qualidade dessa medição que vai aparecer na odometria.

Utilize régua, paquímetro ou trena. Meça cada grandeza ao menos 3 vezes, anote todas as medidas e use a média:

- Raio da roda (r): meça o diâmetro da roda esquerda e da direita e divida por 2. As duas são iguais?
- Distância entre as rodas (L): distância entre os pontos de contato das duas rodas com o chão (de centro a centro da banda de rodagem), medida perpendicularmente ao sentido de deslocamento.
- Deslocamento do LiDAR (opcional, Tarefa 2.6.4): distância do eixo das rodas até o centro do LiDAR, ao longo do eixo longitudinal do robô.

Registre os valores com unidade e incerteza estimada (ex.: r = 33,0 ± 0,5 mm). Eles serão usados nas Etapas 1 a 4 e devem constar no relatório.

2.3 Etapa 1 — Conexão e assinatura dos tópicos

Importe suas funções, conecte-se ao robô e assine os três tópicos que serão usados: /joint_states (posição das rodas), /odom (odometria do robô) e /scan (LiDAR).

```
import time, math, threading
import numpy as np
import matplotlib.pyplot as plt
import roslibpy
from lab2_utils import *

ROBOT_IP = '192.168.0.XXX'   # <-- IP do seu robô
NS = ''                      # prefixo dos tópicos, ex: '/tb03' (só se o professor
indicar)

# Parâmetros geométricos: MEDIDOS por vocês na seção 2.2.1 (em metros)
R_RODA = 0.0   # <-- raio da roda medido
L_EIXO = 0.0   # <-- distância entre as rodas medida
```

```
assert R_RODA > 0 and L_EIXO > 0, 'Preencha R_RODA e L_EIXO com os valores medidos!'

client = roslibpy.Ros(host=ROBOT_IP, port=9090)
client.run()
print('Conectado:', client.is_connected)

# --- estado compartilhado entre os callbacks ---
phi_prev   = [None]          # última leitura das rodas (esq, dir)
pose_prop  = [0.0, 0.0, 0.0] # sua odometria (x, y, theta)
traj_prop  = []              # histórico da sua odometria
traj_odom  = []              # histórico de /odom: (x, y)
ultimo_scan = [None]

def cb_joint(msg):
    i_e = msg['name'].index('wheel_left_joint')
    i_d = msg['name'].index('wheel_right_joint')
    phi_e, phi_d = msg['position'][i_e], msg['position'][i_d]
    if phi_prev[0] is not None:
        dphi_e = phi_e - phi_prev[0][0]
        dphi_d = phi_d - phi_prev[0][1]
        x, y, th = pose_prop
        pose_prop[:] = atualiza_odometria(x, y, th, dphi_e, dphi_d, R_RODA, L_EIXO)
        traj_prop.append(tuple(pose_prop))
    phi_prev[0] = (phi_e, phi_d)

def cb_odom(msg):
    p = msg['pose']['pose']['position']
    traj_odom.append((p['x'], p['y']))   # usaremos apenas a posição

def cb_scan(msg):
    ultimo_scan[0] = msg

t_joint = roslibpy.Topic(client, NS + '/joint_states', 'sensor_msgs/JointState')
t_odom  = roslibpy.Topic(client, NS + '/odom', 'nav_msgs/Odometry')
t_scan  = roslibpy.Topic(client, NS + '/scan', 'sensor_msgs/LaserScan')
t_joint.subscribe(cb_joint)
t_odom.subscribe(cb_odom)
t_scan.subscribe(cb_scan)

time.sleep(2)
print('joint:', len(traj_prop), '| odom:', len(traj_odom), '| scan:', ultimo_scan[0]
is not None)
```

Nada chegou?

Se alguma contagem for zero, verifique o NS (prefixo dos tópicos) e a conexão. Chame o monitor se persistir.

2.4 Etapa 2 — Trajetória livre e sua odometria

Agora o grupo define uma trajetória livre, combinando trechos retos e curvas à vontade (sem precisar fechar um percurso). Sugestão: pelo menos 3 trechos, com 20 a 40 segundos no total. Posicione o robô em um espaço livre de cerca de 1,5 m x 1,5 m e marque no chão, com fita, a posição e a direção iniciais. A trajetória é definida editando a sequência de chamadas de move dentro da função trajetoria().

```
cmd_vel = roslibpy.Topic(client, NS + '/cmd_vel', 'geometry_msgs/Twist')

def move(v, w, duracao, freq=10):
    """Publica (v, w) a 'freq' Hz durante 'duracao' segundos e para."""
    msg = roslibpy.Message({'linear': {'x': v, 'y': 0.0, 'z': 0.0},
```

```
                              'angular': {'x': 0.0, 'y': 0.0, 'z': w}})
    t0 = time.time()
    while time.time() - t0 < duracao:
        cmd_vel.publish(msg)
        time.sleep(1.0 / freq)
    cmd_vel.publish(roslibpy.Message({'linear': {'x': 0.0, 'y': 0.0, 'z': 0.0},
                                      'angular': {'x': 0.0, 'y': 0.0, 'z': 0.0}}))
    time.sleep(0.3)

def trajetoria():
    """Trajetória do grupo: EDITE livremente (v em m/s, w em rad/s, duração em s)."""
    move(0.1, 0.0, 4)    # reta
    move(0.0, 0.4, 3)    # giro no lugar
    move(0.1, 0.3, 5)    # curva
    move(0.1, 0.0, 3)    # reta

# Marca o início do experimento
traj_prop.clear(); traj_odom.clear(); pose_prop[:] = [0.0, 0.0, 0.0]
trajetoria()

P = np.array(traj_prop)
plt.figure(figsize=(5, 5))
plt.plot(P[:, 0], P[:, 1], label='odometria própria')
plt.axis('equal'); plt.grid(True); plt.legend()
plt.xlabel('x [m]'); plt.ylabel('y [m]'); plt.show()
```

Tarefa 2.4.1: descreva a trajetória que seu grupo definiu e verifique se o gráfico condiz com o que o robô fez no chão. Meça com a fita a posição final real do robô em relação ao ponto de partida e anote o valor.

2.5 Etapa 3 — Comparação visual com /odom

Compare visualmente a trajetória calculada por você com a publicada em /odom. Usaremos apenas a posição (x, y) de /odom. Como a origem de /odom é o ponto onde o robô foi ligado, deslocamos a curva para começar em (0, 0). Os eixos de /odom também podem estar girados em relação aos da sua odometria; isso é esperado, então compare principalmente o formato das trajetórias, e não a orientação dos eixos.

```
O = np.array(traj_odom)
O = O - O[0]          # começa na origem

fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].plot(P[:, 0], P[:, 1]); ax[0].set_title('Odometria própria')
ax[1].plot(O[:, 0], O[:, 1], 'r'); ax[1].set_title('/odom')
for a in ax:
    a.axis('equal'); a.grid(True); a.set_xlabel('x [m]'); a.set_ylabel('y [m]')
plt.show()
```

Tarefa 2.5.1: as duas trajetórias têm o mesmo formato e o mesmo tamanho aproximado? Anote as diferenças que você observa e compare as duas posições finais com a posição final real medida na Tarefa 2.4.1.

Tarefa 2.5.2: liste ao menos duas razões possíveis para as diferenças (pense em: erros na medição de r e L, taxa de amostragem, escorregamento das rodas).

Tarefa 2.5.3: repita com velocidades maiores (por exemplo, dobre v e w) e observe visualmente se a diferença entre as duas trajetórias aumenta.

2.6 Etapa 4 — Mapa métrico (nuvem de pontos)

Monte um pequeno cenário com 2 ou 3 caixas e/ou uma parede de papelão dentro da área de teste. Vamos fazer o robô repetir a trajetória definida pelo grupo enquanto, em paralelo, cada scan do LiDAR é transformado para o referencial global usando a pose atual da sua odometria e acumulado em uma nuvem de pontos.

```
def mapear(passo=3, freq=5):
    """Roda a trajetória em uma thread e acumula o mapa enquanto o robô anda."""
    nuvem = []
    th = threading.Thread(target=trajetoria)
    th.start()
    while th.is_alive():
        s = ultimo_scan[0]
        if s is not None:
            pts = scan_para_mundo(s['ranges'], s['angle_min'], s['angle_increment'],
                                  tuple(pose_prop), passo=passo)
            nuvem.append(pts)
        time.sleep(1.0 / freq)
    return np.vstack(nuvem)

traj_prop.clear(); pose_prop[:] = [0.0, 0.0, 0.0]
mapa = mapear()

P = np.array(traj_prop)
plt.figure(figsize=(6, 6))
plt.scatter(mapa[:, 0], mapa[:, 1], s=2, c='k', label='pontos do LiDAR')
plt.plot(P[:, 0], P[:, 1], 'r-', label='trajetória (odometria própria)')
plt.axis('equal'); plt.grid(True); plt.legend(); plt.show()
```

Tarefa 2.6.1: o mapa reproduz o formato do cenário (caixas, paredes)? Há paredes ou quinas duplicadas? Meça com a fita a distância real entre dois objetos do cenário e compare com a distância no mapa. Salve a figura para o relatório.

Tarefa 2.6.2: altere o valor de L_EIXO em +10% (sem mexer no robô), repita o mapeamento e compare com o mapa anterior; depois restaure o valor medido e faça o mesmo alterando R_RODA em +10%. Qual parâmetro afeta mais a forma do mapa? Qual afeta mais a escala? Por quê? O que isso diz sobre a importância de medir bem os parâmetros?

Tarefa 2.6.3: com o robô parado em frente a uma parede, gire-o no lugar com w = 0,3 rad/s e depois com w = 1,0 rad/s, acumulando o mapa. O que acontece com a parede no mapa? Por que a sincronização entre a pose e o scan (que chegam em tópicos diferentes, com a pose do último valor recebido) importa mais em velocidades altas?

Tarefa 2.6.4 (opcional): o LiDAR não está exatamente sobre o centro do eixo das rodas. Use o deslocamento que você mediu na seção 2.2.1, incorpore-o em scan_para_mundo e verifique se o mapa melhora.

Tarefa 2.6.5 (opcional): reduza o número de pontos do mapa descartando pontos a menos de 2 cm de algum ponto já existente (filtro de voxel) e compare a densidade e o tempo de plotagem.

2.7 Encerramento

Ao finalizar, pare o robô, cancele as assinaturas e encerre a conexão:

```
move(0.0, 0.0, 0.2)
t_joint.unsubscribe(); t_odom.unsubscribe(); t_scan.unsubscribe()
client.terminate()
```

Antes de sair

Devolva o robô ao estado indicado pelo monitor, desfaça o cenário e guarde as fitas e caixas utilizadas.

2.8 O que entregar

1. O arquivo lab2_utils.py com as três funções implementadas.
2. Figuras: comparação visual da trajetória própria com /odom (Etapa 3) e os mapas gerados nas Tarefas 2.6.1 e 2.6.2.
3. Os parâmetros medidos (r, L e, se feito, deslocamento do LiDAR), com unidade, repetições e incerteza; a descrição da trajetória escolhida (Tarefa 2.4.1) e as respostas às Tarefas 2.5.1, 2.5.2, 2.5.3 e 2.6.3.
4. Um parágrafo (5 a 8 linhas) discutindo: por que o mapa construído apenas com odometria tende a se deformar ao longo do tempo, e que tipo de informação adicional poderia corrigir isso? (essa pergunta antecipa os temas de Localização e SLAM).